Lecture Notes - Day 9 (Evening)

Password Hashing, Login API and JWT Authentication in Express.js

Instructor: Abhishek Purohit | Date: 29 Oct 2026 | Prepared for: Sumeet Sharma

What this lecture covers (in teaching order)

1. Recap: why APIs must be secured, and why admin **registration** must exist before **login**
2. The plain-text password problem and the fix: hashing passwords with **bcrypt** (**hash** vs **hashSync**)
3. Testing registration (Suresh, Nitin), widening the password column, and why the same password gives different hashes
4. Building the **login API**: find admin by phone, then compare the password with **compareSync**
5. **JWT** (JSON Web Token): header, payload, signature, secret key, restaurant analogy, jwt.io
6. The **jsonwebtoken** package: **jwt.sign()** to create a token; secret keys, RSA keys and key rotation
7. Where the client should send the token: URL vs body vs **request header**; the **Authorization: Bearer** format
8. The **verifyToken** middleware: read the header, split the token, **jwt.verify()**, 401 responses, **next()**
9. Securing the product routes and testing the whole flow in Postman
10. Real-world authentication beyond this lecture: expiry, RBAC, refresh and access tokens, OAuth
11. End-of-day announcements (exam and project logistics)

How to read these notes

- Normal text is taken from the lecture and converted from spoken words into clear written English.
- Boxes labelled **[Extra - not from the lecture]** are my additions (extra explanation, improvements, interview material). Treat them as supplementary.
- Grey boxes show **code**; green boxes show **expected output**; red boxes show **common mistakes**.
- The transcript is auto-generated and partly garbled (mixed Hindi/English). Where a sentence was unclear I used the meaning that the code and the surrounding explanation make certain.

Note: You supplied the final **verifyToken** middleware, **AdminController** (save + login) and **AdminRouter**. Those are used as given (only a few very long comment lines are wrapped so they fit the page). The **ProductRouter** with the middleware, the intermediate (earlier-stage) versions of the code, and the **index.js** were not supplied: they are **reconstructed** from what the instructor described, and marked as such.

1. Recap: Why We Need Login Authentication

Currently every API of the product application is open. Anyone who knows the URL can call it, whether they are an admin, a user, or just a random person, and whether or not they are a registered entity of your application. The instructor said there has to be a restriction: **only a logged-in person** can call these APIs.

That is why the application is being secured with **login authentication**. The entity created for login is the **admin**. The scenario to build:

1. First the admin must **register** (if there is no admin data in the database, nobody can log in). The registration API was built in the morning session.
2. The admin then **logs in**. On success the server sends back a **token**, which is the proof of login.
3. For every further call to a secured API (delete product, update product, create product, fetch products) the admin must send that token in the request.
4. If the token is missing (or not valid), the server rejects the request and replies that the token is missing / please log in first / the user is not authenticated.

This lecture implements steps 1 to 4 in code: secure password storage, the login API, token generation, and the token-checking middleware.

2. Storing Passwords Securely with bcrypt

Phase 1 - Initial approach: plain-text password in the database

In the morning the admin registration API inserted the password exactly as the client sent it. The instructor's warning:

> Never ever store passwords, or any secret data, as plain text inside your database. It will be visible to every developer who can open the database. We do not want to expose the password of the admin or of any user.

Phase 2 - Problem and the idea of a stronger encryption

- The fix is to **encrypt the password first** and only then insert it into the database.
- But a simple encryption has a problem: anyone can easily **decrypt** it again. That must not be possible.
- So we need an algorithm that cannot be reversed. The instructor reminded the class of algorithms they may have heard of in school or college: **SHA-256** and **bcrypt**. There are many such algorithms for protecting password values.
- We will not write the algorithm from scratch (it would take too much time). We use a **third-party module** with a ready implementation. The module is **bcrypt**.

[Extra - not from the lecture] Hashing vs encryption. What bcrypt does is technically **hashing**: a one-way transformation. Encryption (for example AES) is two-way: whoever has the key can decrypt. The lecture says 'encrypt' throughout; the behaviour it describes (cannot be decrypted back) is hashing. Because of this, a login check does not decrypt the stored password; it hashes the typed password again and compares (section 3).

Concept / Question	Definition or explanation	Purpose / key difference
Plain text password	The password stored exactly as typed	Dangerous: anyone with DB access can read it
Encryption	Scrambling data so it can be unscrambled with a key	Reversible, so not suitable for passwords (extra)
Hashing (bcrypt)	One-way conversion of a password to a long string	Cannot be turned back; used for storing passwords (extra)
bcrypt	Third-party npm module with password hashing functions	Takes plain password, returns hashed string

Phase 3 - Improved approach: bcrypt

Step 3.1 - Install and import

```
npm install bcrypt        # the lecture installed version 6

import { compareSync, hashSync } from "bcrypt";
```

The module gives functions that take the plain-text password as input and return the encrypted (hashed) password. Two variants exist for each job:

Function	Behaviour	Used for in the lecture
`hash(plain, rounds)`	**Asynchronous**: the rest of the code keeps running while the password is being processed	Not used
`hashSync(plain, rounds)`	**Synchronous**: the code waits, then returns the hashed string	**Used.** We must not insert the data until the password has been hashed
`compare(plain, hash)`	Asynchronous comparison	Not used
`compareSync(plain, hash)`	Synchronous comparison; returns true or false	Used in the login API (section 3)

Instructor's reasoning: after the password is encrypted we want to insert the data. The insert must happen **only once** the password has been encrypted, so we need synchronous behaviour, which is why **hashSync** is chosen.

Arguments of hashSync: the first is your plain-text password string. The second can be either a key or a **number of rounds**; the instructor passed **10** and said, in simple terms, that the password is encrypted ten times and the final encrypted value is returned as a string.

[Extra - not from the lecture] The number is the bcrypt **cost factor** (also called salt rounds). It means the internal hashing work is repeated 2^{10} = 1024 times, which makes brute-force guessing slow. bcrypt also adds a random **salt** automatically. Higher number = safer but slower (10 is a common default).

Step 3.2 - Updated save (register) function

Purpose: register a new admin, but store a hashed password. **Change compared with the morning version:** two additions (the import and the **hashSync** line) and the hashed value is inserted instead of the plain one.

```
// src/controllers/AdminController.js  (save function)
import { compareSync, hashSync } from "bcrypt";
import { sqlConnectionPool } from "../configs/DbConfig.js";
import jwt from 'jsonwebtoken';

export async function save(request, response) {
    try {
        // step 1: get data from client
        const { name, phone, password } = request.body;

        const encryptedPassword = hashSync(password, 10); //async

        // step 2: create a SQL INSERT query
        // const insertQry = `INSERT into product(name, price, description, qty)
        //     VALUES('${name}', ${price}, '${description}', ${qty})`
        // this above approach (concating data directly in the SQL query)
        // is prone to sql injection attack

        const insertQry = `INSERT into admin(name, phone, password) VALUES(?,?,?)`;

        // step 3: run this query in the db
        const [result] = await sqlConnectionPool.query(
            insertQry, [name, phone, encryptedPassword]
        );
        console.log(result.insertId);

        response.send({ message: "admin registered", insertedId: result.insertId });
```

```
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}
```

Explanation

Line	What it does and why
`hashSync(password, 10)`	Takes the plain password from the request body and returns the hashed string. Stored in **encryptedPassword**.
`[name, phone, encryptedPassword]`	The third placeholder now receives the **hashed** value. The plain password is never inserted, so the database never holds it.
`VALUES(?,?,?)`	Three placeholders for three columns (id is auto-increment). Parameterised query, safe from SQL injection.
`result.insertId`	The id MySQL generated for the new row; returned to the client.

Note: The comment **//async** next to **hashSync** in the supplied code is misleading: **hashSync** is the **synchronous** version. The asynchronous one is **hash**. Also, the 'step 1' and insert comments are copied from the product controller and still mention product fields; they are leftovers.

Step 3.3 - Testing the registration

The instructor ran **node index.js** and used Postman: **POST** to `/admin`, body type raw / JSON.

```
POST /admin
{
  "name": "Suresh",
  "phone": "808080",
  "password": "Suresh123"
}
```

```
{ "message": "admin registered", "insertedId": 1 }
```

Then `SELECT * FROM admin;` in MySQL shows a long, unreadable string in the password column instead of Suresh123. Because encrypted values are long, the instructor checked the column length (**500**) and widened it to **1000** so a longer encrypted value can never be truncated or cause an error:

```
DESCRIBE admin;
ALTER TABLE admin MODIFY password VARCHAR(1000);
DESCRIBE admin;
```

A second admin was then registered the same way (name **Nitin**, phone **909090**, password **Nitin123**) and the select query showed another encrypted value.

[Extra - not from the lecture] A bcrypt hash looks like `$2b$10$.....` (illustration, not a real value). It is always about **60 characters**, so 500 was already sufficient; widening to 1000 is just a safe margin.

Always remember: even if two users have the same password, their encrypted values are totally different. The encryption of Suresh123 for one user is not the same as the encryption of the same password for another.

[Extra - not from the lecture] Why: bcrypt generates a new random salt for every hash. This is a feature: if one hash leaks, it does not reveal who else uses the same password. It is also why login must use **compareSync** and not simple equality (section 3).

Common mistake: Do not re-run an old plain-text insert and expect login to work. Rows inserted **before** hashing was added contain plain passwords, so compareSync will fail for them. Delete or re-register those admins.

3. The Login API

3.1 Purpose and flow

The second operation for the admin is **login**. The login API receives the credentials, checks them against the database, and on success gives the admin a **token** as proof of login (the token comes in section 4/5; first the credential check is built).

On the front end there would be a login form with two input fields. In this lecture the fields are **phone number** and **password** (in other applications it can be email, or username, and password). The instructor compared this to what Flipkart and similar sites do behind the login button: call a login API that compares what you typed with the database values.

> The instructor's study advice (on being told AI can write the implementation): understand the **behaviour** well, not only the implementation. If you rely only on someone else to write it, you cannot call yourself knowledgeable.

3.2 The problem: plain password vs hashed password

The instructor asked the class: can we compare the plain-text password from the request with the database value directly in an SQL query? The class answered **no**: the database holds the encrypted value but the request holds plain text, so they can never be equal.

The plan therefore has two parts:

1. **Find the admin by phone number** with a SELECT query. The phone number can be compared directly in SQL.
2. If an admin exists, **compare the password in code** using bcrypt's compare function (it hashes the plain password and checks it against the stored hash).

Phase 1 - First stage: find the admin and look at the result

The instructor wrote the query first and sent a **dummy response**, just to look at what the database returns. (Reconstructed from the description.)

```
export async function login(request, response) {
    try {
        const { phone, password } = request.body;

        const loginQry = "SELECT * FROM admin WHERE phone = ?";
        const [result] = await sqlConnectionPool.query(loginQry, [phone]);

        console.log(result);                        // look at the database result
        response.send({ message: "login process" });   // dummy response

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}
```

- **Where does this go?** A new exported function **login** in AdminController.js with the usual **request, response** arguments and the same try-catch block (the instructor copied it from save).
- The query uses a **?** placeholder with the phone in an array to avoid SQL injection. **await** needs **async** on the function.
- A SELECT always returns an **array of rows**. We destructure the first element as **result**.

Route for login

```
// src/routers/AdminRouter.js
import { Router } from "express";
import { login, save } from "../controllers/AdminController.js";

const adminRouter = Router();

adminRouter.post("/admin", save);
adminRouter.post("/admin/login", login);

export default adminRouter;
```

The full login URL is `/admin/login`. The admin router was already registered in index.js, so only a server restart is needed. In Postman: POST to /admin/login with a JSON body containing **phone** and **password** (at this stage any password value is sent, since only the phone is compared).

What the console shows

Phone sent	Console output (result)	Meaning
A phone that exists	An array with **one object** (the admin row, including the hashed password)	Phone number matched
A phone that does not exist	A **blank array** `[]`	No admin has this phone number

Phase 2 - Second stage: handle both cases and check the password

Now the real logic. **result** is always an array, so:

- **result.length === 0**: no admin with that phone. Send an early response: the phone number is not valid. (The instructor said you may add a status such as 404 or 400, 'totally up to you'; the supplied code sends the message without a status.)
- Otherwise the phone matched. Take the admin object with **result[0]**. The instructor assumed the phone is unique, so there is at most one object.
- Compare the passwords with **compareSync(plainPassword, adminData.password)**. It returns a **Boolean**: **true** if they match, **false** if not.
- **true**: login successful. **false**: send 'Password is not valid'.

> We cannot compare equality of a plain password and an encrypted password. Whoever encrypted the password gives a function that compares them: so we use **compareSync**. The plain password comes from the request body; the encrypted password comes from the database as **adminData.password**.

[Extra - not from the lecture] Make **phone** a **UNIQUE** column (`ALTER TABLE admin ADD UNIQUE (phone);`) so the assumption 'phone is unique' is enforced by the database. Also, for security, many applications return one generic message ('invalid credentials') for both a wrong phone and a wrong password, so attackers cannot tell which one was wrong. The lecture used two separate messages.

Testing the login (token not yet added)

Request body	Response	Why
Phone that does not exist	`{"message":"Phone no. is not valid"}`	result is an empty array
Correct phone 808080, wrong password	`{"message":"Password is not valid"}`	Phone matched, compareSync returned false
Correct phone 808080, password Suresh123	`{"message":"Login successful"}`	Both matched. (Token is added in section 5)

The instructor noted that if Postman seems to give a stale result, open a new request tab (he suspected caching) and resend.

4. JWT: JSON Web Token

Every time the admin wants to use secured APIs, he logs in (maybe every 24 hours or every two days, depending on the timeline you set). So after a successful login the server must **generate a token** and send it in the response. How does that token look and how is it built?

4.1 What is a JWT?

> JWT stands for **JSON Web Token**. A token generated at the back end and shared with the client, built from JSON data, is called a JSON Web Token.

Why is JSON in the name? Because at the back end, when you create the token, the token can **contain data**, and that data is a JSON object (key/value pairs). Examples of what a payload can contain: the **id of the logged-in admin**, the **time of login**, the **expiry time** of the token, the **user role**, or any other data you want.

4.2 The three parts of a JWT

Part	What it contains	Who handles it
Header	The details of the **algorithm** used to encrypt the token (and the token type)	Set **automatically**; you do not deal with it
Payload	The **data** you want inside the token (admin id, login time, expiry, role ...)	**You** specify it when creating the token
Signature	Made using the **secret key** used to sign the JWT	Set automatically from the secret key you pass

The instructor showed the official site **jwt.io**. On the left is a sample JWT: a long alphanumeric string beginning with **eyJ**. The dots in it separate the three parts. As a developer you only worry about the **payload**; the library reads your payload, adds the header and signature, and gives back one alphanumeric string (it looks 'encrypted', in the way a password hash does).

```
xxxxxxxxxx . yyyyyyyyyy . zzzzzzzzzz        (a JWT has exactly two dots)
   header      payload      signature
(algorithm)   (your data)  (made with the secret key)
```

> **[Extra - not from the lecture]** Important correction to 'encrypted': the header and payload of a JWT are only **Base64-encoded**, not secret. Anyone holding the token can decode and read the payload (try pasting a token into jwt.io). So **never put a password or other secret in the payload**. What the secret key protects is the **signature**: nobody can change the payload without breaking the signature.

4.3 Why the signature and secret key matter (restaurant analogy)

Recall the self-service restaurant from the morning. The instructor (as restaurant owner) hands out payment tokens. If a customer makes a **duplicate token** that looks identical, the food counter must be able to tell. So the restaurant puts something on each real token that only the owner knows about: a secret mark, which the owner checks by shining a torch light on it. A duplicate token will not have that mark, so it is invalid.

In the same way a **secret key** is used while creating the token. Later, when a request arrives with a token, the server uses the **same key** to check that the token was really created by this server and is not duplicate or invalid.

Restaurant	JWT
Token handed out at the payment counter	Token generated by login API with jwt.sign()
Secret mark only the owner knows	Secret key used to sign the token
Torch light check at the food counter	jwt.verify() with the same secret key
Duplicate token (no secret mark)	Fake or modified token; verification fails

> While creating a token you specify two things: the **payload** and the **secret key**. The function gives you the JWT; you send it to the client as proof of authentication. Later the server validates the token that comes in the

request using the same secret.

4.4 The jsonwebtoken package

Again, we do not create tokens from scratch. We use a third-party module called **jsonwebtoken** (documented on npmjs.com). Its two main functions:

Function	Purpose	Arguments
`jwt.sign(payload, secret)`	**Creates** a new token	1) payload (JSON object), 2) secret key
`jwt.verify(token, secret, callback)`	**Checks** that a token is valid	1) token string, 2) the same secret key, 3) callback (error, payload)

```
npm install jsonwebtoken
```

After installing bcrypt and jsonwebtoken, package.json lists four dependencies: **bcrypt**, **express**, **jsonwebtoken** and **mysql2**.

4.5 Generating the token in the login API

After the password is confirmed valid, create the token. First import the library with the name **jwt**:

```
import jwt from 'jsonwebtoken';

if (isPasswordValid) {
    const token = jwt.sign({ adminId: adminData.id }, "hello1234");
    response.send({ message: "Login successful", token });
}
```

- `jwt.sign({ adminId: adminData.id }, "hello1234")`: **payload** = a JSON object holding the logged-in admin's database id (so that when we read the token later we can recognise which admin it belongs to; the instructor said you can also add other admin fields like name, but not the password). **Secret key** = `hello1234` (the instructor spoke it as 'hello one two three four').
- `response.send({ message, token })`: **token** is shorthand for `token: token` (JavaScript object shorthand). The JSON response now carries the token.

```
{
  "message": "Login successful",
  "token": "eyJhbGciOi... (a long three-part string)"
}
```

Postman results: invalid phone gives 'Phone no. is not valid'; correct phone with wrong password gives 'Password is not valid'; correct phone and password (808080 / Suresh123) gives 'Login successful' **plus the token**, the actual proof of login. The client stores this token safely and sends it with every secured API call.

[Extra - not from the lecture] jwt.sign() also adds an `iat` (issued-at) value to the payload automatically. Without an expiry the token **never expires**. The lecture mentioned expiry as something a payload can contain; to add it use the third argument: `jwt.sign({ adminId: adminData.id }, secret, { expiresIn: "1h" })`. After the time passes, jwt.verify reports an error and the middleware answers 401, so the admin logs in again.

4.6 Secret keys in real applications

`hello1234` is a weak, hard-coded key used only for learning. The instructor explained what real applications do:

- Simple keys like 'hello world' or 'hello1234' are easy to guess. Real systems use randomly generated keys, typically an **RSA key pair**: a **private key** and a **public key** (he showed an online 'RSA private key generator' with a 2048-bit key length).
- The **private key** signs (creates) the token; the **public key** verifies it. Never share the private key.
- Do not hard-code these keys in the source code. If they are in the code, the application is vulnerable. Keep them in the environment (for example a **.env** file, or environment variables when deployed with **Docker** or **Kubernetes**) and have the application read them at start-up.

- **Rotate** (change) the keys at regular intervals, for example every week or every fortnight. If someone gets the keys they can generate tokens for any admin or customer.
- For now, as a beginner, the lecture continues with the simple hard-coded secret.

> **[Extra - not from the lecture]** The supplied code uses a **symmetric** secret (one key for both sign and verify). With the RSA pair the same idea applies, but **sign** uses the private key and **verify** the public key, with the algorithm set to RS256. Reading the secret from `process.env.JWT_SECRET` instead of a literal is the first improvement to make.

5. Securing the APIs with the Token

Now the product APIs must demand the token. Idea: put a **middleware** (the instructor called it **m1**) in front of save, get all, get by id and delete by id (and update, if you have it). The middleware validates the token first and only then lets the controller run.

5.1 Where should the client send the token?

The instructor asked the class the ways of passing data to a server (URL and request body) and discussed each:

Option	Verdict	Reason (instructor)
URL	No	The token is sensitive data. In the URL it is exposed to anyone who can see the URL.
Request body	No	(1) A GET request cannot carry a body, so GET APIs could not receive the token. (2) The body should contain only data related to your **business entity** (for example product data), not other things like tokens.
Request header	**Yes**	A header carries **metadata about the request**. Every request can have one, for any method (get, post, delete ...). The login token is exactly this kind of metadata.

Headers are key/value pairs, like the response headers you may have seen. Another example of metadata that belongs in a header: the **language** in which the client wants the response. That is not business data, so it goes in a header. In Postman, the Headers tab already shows default headers (Content-Type, Content-Length, Accept-Encoding, ...) which are sent automatically; we can add our own. In real front-end code these (method, body, headers) are all set programmatically.

5.2 The Authorization header and the Bearer format

There is a **predefined header named Authorization** in the HTTP specification, so we do not invent our own. The key is **Authorization**. The value must follow the format:

```
Authorization: Bearer <token>
               ^^^^^^ ^^^^^^^
               type   the JWT from the login response (one space between)
```

The authorization **type** comes first, then a space, then the credential. There are different types (**Basic**, **Digest**, **Bearer**). For a JWT the type must be **Bearer**. (Basic and Digest are for other credentials such as username/password; look them up in your free time.) Postman also has an **Authorization** tab where you can choose the Bearer Token type and paste the token.

Complete client flow: call the login API, copy the token from the response, then for each secured API add the header `Authorization: Bearer <token>`. Without it the request is rejected.

5.3 The verifyToken middleware

File: **src/middlewares/verifyToken.js** (the middlewares folder was already created in the morning). A middleware function takes **three** arguments: **request, response, next**. (Reminder: middleware = code that runs **before** the controller and decides whether the controller should run.)

Phase 1 - First stage: only check that the header exists

Reading data from the request depends on where it was sent:

Where data is sent	How to read it
URL path parameter	`request.params`
Request body	`request.body`
Request header	`request.get('HeaderName')`

The first version only checked that an Authorization header exists, and otherwise answered 401 'token is missing'. (Reconstructed from the description.)

```
export function verifyToken(request, response, next) {
    try {
        const authHeader = request.get('Authorization');   // "Bearer ey..."

        if (authHeader) {
            next();                    // header exists -> go to the controller
        }
        else {
            response.status(401).send({ message: "Token is missing." });
        }
    } catch (error) {
        console.log(error);
        response.send({ message: "Something went wrong" });
    }
}
```

Status 401 (Unauthorized) is used because it is the **client's fault**: the client did not send credentials. The message can be anything meaningful, such as 'token is missing' or 'please log in first'.

Then the middleware is added to the product routes. In the router the middleware goes **between the path and the controller** (reconstructed ProductRouter):

```
// src/routers/ProductRouter.js   (reconstructed)
import { Router } from "express";
import { save, getAll, getById, deleteById } from "../controllers/ProductController.js";
import { verifyToken } from "../middlewares/verifyToken.js";

const productRouter = Router();

productRouter.post("/product", verifyToken, save);
productRouter.get("/product", verifyToken, getAll);
productRouter.get("/product/:id", verifyToken, getById);
productRouter.delete("/product/:id", verifyToken, deleteById);

export default productRouter;
```

Express runs the functions left to right: **verifyToken** first, then the controller only if **next()** was called.

What the instructor demonstrated at this stage:

Request	Result
GET /product with **no** Authorization header	'Token is missing.' (server blocks access)
GET /product with Authorization = **any random value**	Product data is returned, because stage 1 only checks that the header **exists**, not that it is valid

The second row shows the weakness: **presence is not validity**. That is why the next stage is needed.

Phase 3 - Final version: verify the token with jwt.verify

Now the token is read from the header and checked using the **same secret key** used at sign time.

```
import jwt from 'jsonwebtoken';

export function verifyToken(request, response, next){
    try {
        // request.params
        // request.body
        const authHeader = request.get('Authorization'); // "Bearer ey..."
```

```
        if(authHeader){
            const token = authHeader.split(" ")[1];// ["Bearer","ey...."]
            jwt.verify(token, "hello1234", (error, payload)=>{
                if(error){
                    response.status(401).send({message:"Token is not valid."});
                }
                else{
                    next();
                }
            });
        }
        else{
            response.status(401).send({message:"Token is missing."});
        }
    } catch (error) {
        console.log(error);
        response.send({message:"Something went wrong"});
    }
}
```

Explanation, step by step

Step / code	What happens
`request.get('Authorization')`	Reads the header named Authorization. The value is a string like **"Bearer eyJ..."**, or **undefined** if the header was not sent.
`if (authHeader)`	Header present: continue to validate. Header absent: the **else** branch answers **401 'Token is missing.'**
`authHeader.split(" ")`	Splits the string on the space. Returns an array: **["Bearer", "eyJ..."]**. Always a prefix, a space, then the token.
`[1]`	Index 1 is the second element: the actual JWT. (Index 0 is the word Bearer.) The instructor said authentication only needs this token.
`jwt.verify(token, "hello1234", callback)`	Checks the token's signature with the same secret used in jwt.sign. The result is delivered to the callback.
`(error, payload) =>`	**error** is set if the verification failed (wrong secret, altered/invalid or expired token). **payload** is the data stored at sign time (for us: adminId).
`if (error) ... 401 'Token is not valid.'`	Invalid token: reject with 401; the controller never runs.
`else next()`	Valid token: hand control over to the next function, i.e. the controller.
`catch`	Logs the error for debugging (the instructor said console logging is for local debugging only, not for production) and replies 'Something went wrong' so the client is not left waiting.

```
Client --(request + Authorization: Bearer <token>)--> Router
                                                        |
                                                verifyToken middleware
                          no header ----------------------+--> 401 "Token is missing."
                          invalid token ------------------+--> 401 "Token is not valid."
                          valid token --------next()------+--> Controller --> Response
```

Testing the whole flow in Postman (expected output)

Step	Request	Response
1	POST /admin/login with phone 808080 and password Suresh123	`{"message":"Login successful","token":"eyJ..."}` (copy the token)
2	GET /product, **no** Authorization header	`401 {"message":"Token is missing."}`
3	GET /product, Authorization = **random text** (not Bearer, not a JWT)	`401 {"message":"Token is not valid."}`
4	GET /product, Authorization = **Bearer** + space + token from step 1	The product data (200)

Why step 3 fails: a random value has no space-separated second part, so the token variable is undefined or invalid, and jwt.verify returns an error. **Why step 4 works:** the token was signed with hello1234 and verified with the same key, so no error occurs and next() runs the controller. The same applies to save, get by id and delete by id, which are all protected now. This is JWT authentication.

> With the help of the token we have secured all our APIs: the client must call the login API, get the token, and send it in the Authorization header for every secured API.

Common mistake: Typical mistakes: (1) sending only the token without the word **Bearer** and a space, so split(" ")[1] is undefined; (2) using a **different secret** in verify than in sign; (3) forgetting **next()**, so the request hangs; (4) putting verifyToken **after** the controller in the route; (5) sending the token in the URL or body instead of the header.

[Extra - not from the lecture] Possible improvements (not in the lecture): the callback receives **payload** but the supplied code does not use it. You can store it with `request.admin = payload;` before **next()** so controllers know which admin is calling. The **catch** block should also send `status(500)`. And note that `POST /admin` (registration) and `POST /admin/login` are **not** protected by verifyToken; that is correct for login, but leaving registration open means anyone can create an admin.

5.4 Final code of this session

AdminController.js (save and login, as supplied)

```
import { compareSync, hashSync } from "bcrypt";
import { sqlConnectionPool } from "../configs/DbConfig.js";
import jwt from 'jsonwebtoken';

export async function save(request, response) {
    try {
        // step 1: get data from client {name:"", price: 6677, description:"", qty:8}
        const { name, phone, password } = request.body;

        const encryptedPassword = hashSync(password, 10); //async

        // step 2: create a SQL INSERT query
        // const insertQry = `INSERT into product(name, price, description, qty)
        //     VALUES('${name}', ${price}, '${description}', ${qty})`
        // this above approach (concating data directly in the SQL query)
        // is prone to sql injection attack

        const insertQry = `INSERT into admin(name, phone, password) VALUES(?,?,?)`;

        // step 3: run this query in the db
        const [result] = await sqlConnectionPool.query(
            insertQry, [name, phone, encryptedPassword]
        );
        console.log(result.insertId);

        response.send({ message: "admin registered", insertedId: result.insertId });

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}

export async function login(request, response) {
    try {
        // step 1: get data from client
        const { phone, password } = request.body;

        const loginQry = "SELECT * FROM admin WHERE phone = ?";

        const [result] = await sqlConnectionPool.query(loginQry, [phone]);

        console.log(result);
        if(result.length === 0){
            response.send({message:"Phone no. is not valid"});
        }
```

```
        else{
            const adminData = result[0];
            const isPasswordValid = compareSync(password, adminData.password);
            if(isPasswordValid){
                const token = jwt.sign({adminId: adminData.id}, "hello1234");
                response.send({message:"Login successful", token});
            }
            else{
                response.send({message:"Password is not valid"});
            }
        }

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}
```

Login execution flow (in order)

1. Client POSTs **phone** and **password** to `/admin/login`; the router calls **login**.
2. **login** reads both values from **request.body**.
3. SELECT finds the admin row(s) with that phone.
4. Empty array: reply 'Phone no. is not valid'.
5. Otherwise take **result[0]** and run **compareSync(password, adminData.password)**.
6. false: reply 'Password is not valid'.
7. true: **jwt.sign({ adminId }, secret)** creates the token; reply 'Login successful' with the token.

AdminRouter.js

```
import { Router } from "express";
import { login, save } from "../controllers/AdminController.js";

const adminRouter = Router();

adminRouter.post("/admin", save);
adminRouter.post("/admin/login", login);

export default adminRouter;
```

[Extra - not from the lecture] The complete request flow of the application is now: **Client -> index.js -> Router -> verifyToken middleware -> Controller -> Database** for the protected product APIs; and **Client -> index.js -> Router -> AdminController.login -> Database** for login (no middleware).

6. Real-World Authentication Beyond This Lecture

The instructor stressed that this was a very basic authentication: 'not even the hello world of a real-time application'. In real applications:

Topic	What was said
Token **expiry**	Tokens should expire; the expiry time can be kept in the payload. Keys must be rotated and kept out of the code (section 4.6).
Role-based authentication / RBAC (role-based access control)	Different users have different roles (admin, user ...). The role is placed in the token/payload and the server allows or blocks operations according to the role. It is a major concern in applications of every domain.
Access token and refresh token	Two kinds of tokens are generated. The short-lived **access token** is used to call APIs; the **refresh token** is used to generate a new access token. This is how a site can keep you logged in continuously while the access token expires every 15 or 20 minutes (in answer to a student question about behind-the-scenes logout/login).
OAuth	Instead of building login, token and role handling yourself, you can use the services of OAuth, which handles authentication and role-based control for the application.

> Authentication is an important and sensitive concern of every enterprise application. You should always take it very seriously, because it is what prevents unauthorised access.

The instructor encouraged learning these step by step (it has a learning curve; he faced difficulties himself when he first explored it) and said you should at least be familiar with basic authentication, which is what was built here.

[Extra - not from the lecture] Extra interview note: 'authentication' answers **who are you?** (login, token). 'Authorization' answers **what are you allowed to do?** (roles, RBAC). The header is named **Authorization** but it carries the proof used for authentication.

7. Announcements at the End of the Day

Note: This part of the transcript is from a different speaker (Shilbhushan Khanderao) and is very garbled. The points below are my best reading. **Confirm exact times and rules with the official message from your trainer.**

- **Next day schedule:** reporting time **9:30**; the **MCQ exam** starts at about **10:00**; the **lab exam** follows about half an hour later (questions to be sent by mail around **10:20**). Submission around **12:45** (a 15-minute submission window was mentioned). The lab is about 1 to 1.5 hours. Format is the same as the earlier exam.
- Submission: share your **code and a screenshot** as described; links will be uploaded in the group.
- Trainer was **disappointed that many people have not completed the assignment**; complete it.
- **Project work:** teams will be shared and a Google form will be shared to choose topics. Topics mentioned: authentication, TypeScript, React, and back-end / front-end integration.

8. Source Coverage Checklist

Checked against the transcript from beginning to end:

Topic in transcript	Covered in section
Recap: open APIs, only logged-in admin, registration first, token proof	1
Plain-text password loophole; encryption must not be reversible; SHA-256, bcrypt; third-party module	2
npm install bcrypt; hash vs hashSync; arguments (rounds = 10); encryptedPassword inserted	2 (Steps 3.1 - 3.2)
Postman test (Suresh, Nitin), column widened to 1000, same password gives different hash	2 (Step 3.3)
Login controller; phone + password; why plain vs hashed cannot be compared in SQL	3.1 - 3.2
Select by phone, dummy response, route /admin/login, empty array vs one object	3 (Phase 1)
result.length, result[0], compare vs compareSync, isPasswordValid, messages; Postman tests	3 (Phase 2)
JWT meaning, JSON data, header/payload/signature, jwt.io, three parts with dots	4.1 - 4.2
Signature and secret key, restaurant analogy (duplicate token, torch light)	4.3
jsonwebtoken, sign and verify, npm install, package.json	4.4
jwt.sign in login, payload adminId, secret key, token in response, Postman test	4.5
RSA private/public keys, env variables, Docker/Kubernetes, key rotation	4.6
Where to send token: URL vs body vs header; metadata; language example; Postman headers	5.1
Authorization header; Bearer vs Basic vs Digest; format Bearer space token	5.2
verifyToken: request.get, split, 401 token missing, route placement, stage tests	5.3 (Phases 1)
jwt.verify with callback, error/payload, 401 not valid, next(), full Postman test	5.3 (Phase 3)
Basic auth only; expiry, RBAC, refresh/access tokens, OAuth; student question on auto logout	6
Exam / project announcements	7

9. Revision Material

9.1 Summary of major topics

- **Never store plain-text passwords.** Use bcrypt: **hashSync(password, 10)** before inserting; the hash differs even for identical passwords.
- **Login API:** read phone + password; SELECT by phone; empty result = invalid phone; **compareSync(plain, storedHash)** for the password; on success create and return a token.
- **JWT** = JSON Web Token: **header.payload.signature**. You provide the payload; the secret key creates the signature.
- **jwt.sign(payload, secret)** creates a token; **jwt.verify(token, secret, callback)** checks it.
- Secrets must not be hard-coded in real applications; use RSA keys / environment variables and rotate them.
- Send the token in the **Authorization header** as **Bearer <token>**, not in the URL or body.
- **verifyToken middleware:** no header = 401 'Token is missing.'; invalid token = 401 'Token is not valid.'; valid = **next()**.
- Real systems add expiry, RBAC, refresh/access tokens, or use OAuth.

9.2 Important definitions and syntax

Concept / Question	Definition or explanation	Purpose / key difference
bcrypt	npm module for password hashing	Protects stored passwords
`hashSync(plain, 10)`	Synchronous; returns the hash string	Used before the INSERT
`compareSync(plain, hash)`	Synchronous; returns true / false	Checks the password at login
JWT	JSON Web Token with three dot-separated parts	Proof of successful login
Header	Algorithm details; automatic	Describes how the token is signed
Payload	Your JSON data (adminId, role, expiry ...)	Identifies who the token belongs to
Signature	Made with the secret key	Detects fake or altered tokens
`jwt.sign(payload, secret)`	Creates a token	Called in the login API
`jwt.verify(token, secret, cb)`	Verifies a token; callback gets (error, payload)	Called in the middleware
`request.get('Authorization')`	Reads a request header	Gets the 'Bearer ...' string
`split(" ")[1]`	Takes the token part after 'Bearer '	Extracts the JWT
401 Unauthorized	Client did not provide valid credentials	Used for missing / invalid token
`next()`	Passes control to the next function	Lets the controller run

9.3 Key differences

Compare	Difference
hash vs hashSync	hash is asynchronous; hashSync is synchronous (waits and returns the value). We use hashSync so the insert happens after hashing.
compare vs compareSync	Same idea: compare is asynchronous, compareSync returns the Boolean directly.
jwt.sign vs jwt.verify	sign creates a token at login; verify validates it on later requests.
Token in URL vs body vs header	URL exposes it; GET has no body and body is for entity data; the header carries metadata and is the right place.
Bearer vs Basic vs Digest	Authorization types. A JWT uses Bearer.
Login API vs verifyToken middleware	Login API checks credentials and issues the token; middleware checks the token on later requests.
Token missing vs token not valid	Missing: no Authorization header. Not valid: header present but jwt.verify returned an error. Both are 401.
Encryption vs hashing (extra)	Encryption is reversible with a key; hashing is one-way. bcrypt is a hash.

9.4 Common mistakes to avoid

Mistake	Correction
Storing the plain password	hashSync first; insert the hash
Comparing plain and hashed password in SQL or with ===	Fetch the row by phone, then use compareSync
Forgetting to import hashSync / compareSync / jwt	import { compareSync, hashSync } from "bcrypt"; import jwt from 'jsonwebtoken';
Password column too short (e.g. old VARCHAR(50))	Use a long column; the lecture used VARCHAR(1000)
Putting secrets (password) in the JWT payload	Payload is readable; store only ids / non-secret data
Hard-coding the secret in real code	Read it from environment variables; rotate keys
Different secret in sign and verify	Use exactly the same secret (or key pair)
Sending token without 'Bearer ' prefix	Authorization: Bearer then a space then the token
Sending the token in URL or body	Use the Authorization header
Only checking that the header exists	Also call jwt.verify
Forgetting next() in the middleware	Call next() when the token is valid
Middleware placed after the controller in the route	Router: path, verifyToken, controller
Old plain-text admin rows still in the table	Delete and re-register so they hold hashes

9.5 Interview questions and answers

Questions marked (E) go beyond what the instructor said; the rest follow the lecture.

Question	Answer
Why should passwords not be stored as plain text?	Anyone with database access can read them. Hash them before saving.
Which module did we use for password hashing?	bcrypt: hashSync(password, 10) to hash and compareSync(plain, hash) to check.
Why hashSync and not hash?	hash is asynchronous; we must have the hashed value before inserting, so the synchronous hashSync is used.
Will the same password give the same hash for two users?	No. The result is different every time (bcrypt adds a random salt).

Question	Answer
Why can't we compare the password inside the SQL query?	The database stores the hash while the request carries plain text; they can never be equal. Fetch by phone, then use compareSync.
What is a JWT?	JSON Web Token: an alphanumeric string with header, payload and signature, given to the client after login as proof.
What does each part of a JWT hold?	Header: algorithm. Payload: your data (admin id, expiry, role). Signature: made from the secret key.
What is the role of the secret key?	It signs the token so the server can later verify it is genuine and not duplicate or altered.
Which functions does jsonwebtoken give us?	jwt.sign() to create a token and jwt.verify() to validate it.
How should secret keys be handled in real applications?	Use randomly generated RSA private/public keys, keep them in environment configuration (not in code), and rotate them regularly.
Where should the token be sent and why?	In the Authorization request header, as Bearer . The URL exposes it, GET has no body, and the body is for entity data.
What is the Bearer scheme?	An Authorization type used for tokens such as JWT, written as 'Bearer' + space + token.
What does the verifyToken middleware do?	Reads the header, extracts the token, verifies it, and calls next() if valid, else sends 401.
Why 401 and not 500?	401 Unauthorized means the client failed to authenticate (client fault); 500 is a server fault.
Why are the login API and middleware different?	Login verifies credentials and issues a token; middleware only validates the token on later requests.
Why is the basic implementation not enough for real applications?	Real systems add token expiry, role-based access control, refresh and access tokens, key rotation, or use OAuth.
(E) Is a JWT encrypted?	A normal signed JWT is encoded and signed, not encrypted; the payload can be read, so keep secrets out of it.
(E) Authentication vs authorization?	Authentication: who you are. Authorization: what you may do.

9.6 Practice exercises (solutions not included)

Easy

1. Install bcrypt and write the import line for hashSync and compareSync.
2. Write the three parts of a JWT in order and say who sets each part.
3. What status code does the middleware send when the Authorization header is missing?

Medium

1. Register two admins with the same password and compare the stored values in MySQL. What do you observe and why?
2. Write a login function that returns different messages for wrong phone and wrong password, and test all three cases in Postman.
3. Add jwt.verify to a middleware and protect the update-product route.
4. Send the token (a) without Bearer, (b) with a wrong secret, (c) correctly. Predict each response, then test.

Challenging

1. Add `expiresIn` to jwt.sign, then test with an expired token.
2. Store the verified payload in the request and use the admin id inside a controller.
3. Move the secret out of the code into an environment variable (.env) and read it in both login and middleware.

4. Design a **role** column for admin (for example ADMIN and MANAGER), put it in the payload, and allow delete only for ADMIN.

9.7 Final revision checklist

[ ] I can explain why plain-text passwords are dangerous.
[ ] I can hash a password with hashSync(password, 10) before inserting it.
[ ] I know the difference between hash/hashSync and compare/compareSync.
[ ] I can build the login API: select by phone, check result.length, compareSync.
[ ] I can explain what a JWT is and its three parts.
[ ] I can explain the role of the secret key using the restaurant analogy.
[ ] I can create a token with jwt.sign and return it in the login response.
[ ] I know why the token goes in the Authorization header and the Bearer format.
[ ] I can write the verifyToken middleware with request.get, split, jwt.verify and next().
[ ] I can attach the middleware to routes and test all four cases in Postman.
[ ] I know that secret keys must not be hard-coded and should be rotated in real applications.
[ ] I know what expiry, RBAC, refresh/access tokens and OAuth are for.